Functions Review

What is a Function?

A function is a self-contained chunk / block of code that performs a specific task. We use functions as a way to create a reusable piece of code, so that we can access it with ease. Let's start by writing a basic function so that we can remember what the syntax looks like.

```
func helloWorld() {
    print("Hello World!")
}

// We call a function by using its name, and then putting parens after
it.
helloWorld()
```

On top of putting code that does something, like print. We can also store constants and variables, or really put any code we want inside this function. However, its important to remember that if we declare a constant or a variable inside a function, we can only access those values while inside that same function. It doesn't exist outside of it.

```
func helloWorldRepeated() {
    let timesToRun = 10

    for time in 1...timesToRun {
        print("Hello World! \(time) times.")
    }
}

helloWorldRepeated()
```

Functions can also take parameters, these parameters are like constants inside our function, but are used to represent data that is passed to our function when it is called.

Syntax for functions that take parameters is very similar to other function syntax, but inside our parens we add our parameter(s). When we add a parameter, we need to give it a name and a `Type`. The `Type` can be anything, `Int`, `Double`, `[String]`, `Dictionary`, Any `Type` that we want it to be. Let's say that we have a function that is meant to take an `Array` of words that would be contained in a sentence, and then its purpose is to print out those words in a single line.

```swift
func myArrayIntoASentence(_ words: [String]) {
    // Variable to hold our sentence as we build it.
    var sentence = ""

    // Use a for-loop to go through all the words we are given.
    for word in words {
        // For each of the words we add them to our sentence.
        sentence += word
        // Add a space between each word.
        sentence += " "
    }

    // After the loop is done, and henceforth our sentence is created.
    // We print it out.
    print(sentence)
}

let wordsInMySentence = ["Hello,", "my", "name", "is", "Lukas"]
myArrayIntoASentence(wordsInMySentence)
```

Let's go back and look at our `helloWorldRepeated` function. Seems like we could improve its usability by adding a parameter to change how many times we print that statement. So let's write the function again that does just that.

```swift
func helloWorldRepeated(times: Int) {
    for time in 1...times {
        print("Hello World Again! \(time) times.")
    }
}

helloWorldRepeated(times: 4)
```

Like variable and constant names, function declarations have to be unique as well. However, you may have noticed that I used the same name, so why does it work? It works because since we added parameters, the declaration is different. Same name, but different declaration. This declaration is called the `Function Signature`. The `Function Signature` does have to be unique, but It's made up of the function's name, its parameters, and its return type, as opposed to just its name.

Function parameters can also have what are called labels. What a label does is it lets us change how we call the function, in order to make the function call more readable. But since it's a label on the parameter name, we don't have to sacrifice the code of the function's readability to do this.

Let's say that we want to change our `helloWorldRepeated` function again to make it more reusable. Instead of it just repeating `"Hello World"` on repeat, we want to make it be able to repeat any sentence we want. How could we do that? We could add another parameter for what we want it to repeat. Remember our functions can take as many parameters as we want them to, we just separate them with a comma.

```
func repeatAndPrint(_ sentence: String, times: Int) {
    // Since we are taking an actual sentence this time,
    // we probably don't want to throw number times into
    // that sentence. So luckily we can just ignore this number.
    // in our loop, so Swift doesnt waste time making that constant for
us.
    for _ in 0..<times {
        print(sentence)
    }
}
```

Our loop and function both look a little different this time, let's go over that.

1. Our function now has two parameters, the first one being a `String` for what we want repeated, the second one being the number of times we want that `String` repeated.
2. We added a function parameter label to our first parameter. Specifically, we added the `_` label, which means that when I call this function, I don't need to write a name or label in the parameters I am passing for that specific parameter.
3. Our `for loop` is also using the `_` in place of a name in the place where we normally would represent whatever value we are on in our loop. This is because we don't need to know what that value is, so we don't want Swift putting in the extra work of representing that value.
4. We started at the number 0 this time. Since we aren't referencing that number it's common to start at 0 as that's where computers start counting by default.
5. I used a non-inclusive range `..<` instead of before where I was using an inclusive range `...`. Since we are starting at 0 we don't want to include the last number of `times` as then we would be doing an extra iteration. So if `times` was 10 it would print out `sentence` 11 times.

Let's do one more function example to show off returning values from a function. If you remember, we can return any `Type` of value from a function. Using the syntax `->` we can specify what `Type` of value our function returns when it is called. We are going to do an example of a function that will count the number of cities that our different states have. We are going to represent our States and their various cities using a `Dictionary`.

```
let statesAndCities = [
    "Utah": ["St. George", "Salt Lake"],
```

```
    "Colorado": ["Denver", "Colorado Springs"]
]

func numberOfCities(in statesCitiesDict: [String: [String]]) -> Int {
    var numberOfCities = 0

    for (_, cities) in statesCitiesDict {
        numberOfCities += cities.count
    }

    return numberOfCities
}
```

Closures

Now that we have recovered functions, let's get into a new topic that builds on functions. Swift treats functions just like other `Types`, such as `Strings` and `Integers`. This means that you can assign a function to a variable, call that function using that variable, pass functions into other functions as a parameter, etc.

When we use functions in that way we call them `Closures`. `Closures` are written differently than normal functions, though we can still use normal functions with `Closures` in the right circumstances.

`Closures` are self-contained blocks of functionality that can be passed around and used like other data in your code.

Let's start with a very basic function.

```
func traveling() {
    print("I'm traveling in my car.")
}
traveling()

// Now let's write a similar function as a `Closure`.
// We are going to store it inside of a Constant.

let driving = {
    print("I'm driving in my car.")
}
driving()
```

Simple, yet effective. Let's look at the `Type` of our `driving` constant. You can see that when we look at the `Type` it is represented like this `() → ()` parens, return symbol,

parens. What does this mean? This is saying that this constant is a `Closure` that takes no parameters and returns nothing. You could also write the declaration like this `() -> Void`, which is a different way to say that you aren't returning anything.

You heard me say that this `Closure` doesn't take parameters and doesn't return anything. This would imply that they could, in fact, have parameters and return values. This is true, and the syntax is similar to functions. Let's write a `Closure` that can take a parameter first. We are going to write it out both with and without `Type Annotation`.

```
let walking = { (place: String) in
    print("I'm going to \(place), by foot.")
}
walking("The Store")

let annotatedWalking: (String) -> Void = { place in
    print("I'm going to \(place), by foot.)
}
annotatedWalking("The School")
```

`closures` are also able to have return values, which means that when we call them they, well, return a value.

```
let returnWalking: (String) -> String = { place in
    return "I'm going to \(place) today!"
}

let message = returnWalking("A Concert")
print(message) // I'm going to A Concert today
```

Congratulations, you are now a professional at making basic `closures` and storing them in `variables` or `constants`. Now we need to apply them in a way that will make them more applicable. I'm talking about being able to pass these `closures` around as parameters to functions!

Higher Order Functions

Since we can treat closures as any other value, we can define them as parameters in another function, or even have a function return a closure. When a function either has a parameter that is a closure, or returns one, it is called a Higher Order Function. Let's do some examples:

```
func travel(action: () -> Void) {
    print("I'm getting ready to go.")
```

```
    action()
    print("I've Arrived!")
}

// If you have a stored closure you can pass it like any other type.
// Just make sure that the closure types are the same!
travel(action: driving)
```

We can also create `closures` inline with the `function`, so that we don't need to create a `constant` just to pass it.

> **Anonymous Functions**
>
> A closure created inline like this is often called an Anonymous Function as it has no name. In Swift most people just call them closures all the time; however, its a common term in other languages!

```
travel(action: {
    print("I'm driving to a place.")
})
```

This kind of usage of `closures` is very common, because of this, swift gives us some spotless ways to do it. Swift has what is called **Trailing Closure Syntax**. If the last parameter to a `function` is a `closure`, we can put the `closure` directly after the parens, letting us skip the label/name of the parameter, and saving us from having to put the closing parenthesis after the `closure`. So, like the following:

```
travel() {
    print("I'm driving to a place.")
}
```

In a situation where a function only takes a closure as a parameter, you can even omit the parentheses entirely!

```
travel {
    print("I'm driving to a place.")
}
```

When working with `closures`, you can get very abstracted to a point of a lot of confusion. So, let's go over a couple of examples of things you can do! First on our list is to show how `closures` that are being used as parameters can take their own parameters.

```
func travelToComicon(travelMethod: (String) -> Void) {
    print("I'm ready to go")
    travelMethod("Comicon")
    print("I've arrived.")
}
```

So when we call this `function` it is asking for a parameter that is a `closure` that takes a parameter. This means that the `function` has to provide a value to pass as the `closures` parameter.

```
travelToComicon { place in
    print("Im going to \(place) in my car.")
}
```

Then to hammer the point home, these closures can also have return values.

```
func travelToEurope(travelMethod: (String) -> String) {
    print("I'm getting ready to go")
    let description = travelMethod("Europe")
    print(description)
    print("I've Arrived")
}

travelToEurope { place in
    return "I'm going to \(place)"
}
```

Let's say that, like above, the `function` we are trying to call required a closure that takes a parameter and returns something. But I want to ignore the parameter. Swift needs to know that the closure you are passing is of the same type. This means we can't just not acknowledge the parameter. We can, however, tell swift that we want to ignore it by replacing the name with an underscore.

```
travelToEurope { _ in
    return "I'm going somewhere"
}
```

We can also return `closures` from `functions`, this is a specific case that I haven't seen used very often in Swift, but because `closures` are just like any other type, it's a possibility.

```
// The first return arrow is saying that this function is returning a
closure.
// The second return arrow is saying what the closure is returning.
func makeMathFunction() -> (Int) -> Void {
    return { number in
        print("\(number) + \(number) = \(number + number)")
    }
}

let mathFunction = makeMathFunction()
mathFunction(2) // 2 + 2 = 4

// We could write the above like this
// but this is just silly looking.
makeMathFunction()(3) // 3 + 3 = 6
```

Why They Are Called Closures

Typically, when we declare a `variable` or `constant` inside a `function`, that `variable` or `constant` gets deleted when the `function` stops running. When we have `closures`, however, they “close” around the values that it access from outside its scope. This is where their name comes from. Let's do an example.

```
// We want to track how many times a closure has been called.
func numberOfTimesIHaveWatched(movie name: String) -> () -> Void {
    var count = 0
    return {
        count += 1
        print("I have watched \(name), \(count) time\(count != 1 ? "s"
: "").")
    }
}
```

Even though, the counter `variable` and the name `constant` are created outside the `closure`, and should disappear when the `function` ends. The `closure` captures those values and keeps them around.

```
var watchShrek = numberOfTimesIHaveWatched(movie: "Shrek")
watchShrek() // I have watched Shrek, 1 time.
watchShrek() // I have watched Shrek, 2 times.
watchShrek() // I have watched Shrek, 3 times.
```

Brief Overview of Closures

1. In swift, `functions` are considered a type, when we use them in this way we call them `closures`. This means that we can pass them around like all the other values we have covered.
2. You can pass `closures` into `functions` as parameters.
3. `closures` can have parameters passed into them.
4. `closures` can return values.
5. You can store as `closure` as a `variable` or `constant`, and call it later.
6. If the last parameter to your `function` is a `closure`, you can use **Trailing Closure Syntax** when you call the `function`.
7. If you use external values inside your `closures`, they will be captured so the `closure` can refer to them later.

More Examples of Closures

```
let add: (Int, Int) -> Int = { val1, val2 in
    return val1 + val2
}

let subtract: (Int, Int) -> Int = {
    return $0 - $1
}

var mathArray = [add, subtract]
let result = mathArray[0](5, 10) // Grab first closure from array and
call it.
print(result)

let multiply: (Int, Int) -> Int = { val1, val2 in
    return val1 * val2
}

mathArray.append(multiply)
```

Above, we created a handful of different `closures`, and then added them to an `Array`. This means that we can iterate over them and call each `closure`.

```
let ten = 10
let four = 4

for operation in mathArray {
```

```
    print(operation(ten, four)
}
// Output would look like the following:
// 14
// 6
// 40
```

Closures and Collection Operations

Swift supports many programming paradigms, including functional programming. One key way that we can see this in Swift are with some common collection operations and closures. Take for example the need to convert all the items in an Array from one type to another. We might write a function like this:

```
func intArrayToDoubleArray(_ nums: [Int]) -> [Double] {
    var doubleValues: [Double] = []

    for num in nums {
        doubleValues.append(Double(num))
    }

    return doubleValues
}
```

Not overly complex, but then what if we also, sometimes, want to then filter out all values over a specific one? We may have to create another function to do that.

```
func removeValues(over upper: Double, from nums: [Double]) -> [Double]
{
    var under: [Double] = []

    for num in nums where num < upper {
        under.append(num)
    }

    return under
}
```

If we wanted to keep performing all these operations our code could get quite large. We could do it all in one for loop, but then it still gets difficult to read. Swift collections provide a large pool of common operations like this using Higher Order Functions. The first example is a map operation, and the second example is a filter operation. If I had an array and I wanted to do both of these operations I could write it out like this:

```
let myStartingIntegers = [1, 2, 10, 20]
let myFinalResult = myStartingIntegers
    .map { Double($0) }
    .filter { $0 ≤ 5 }
```

These chains of functions are great because they are composable, easy to read, and allow us to have nonmutating values!